

COMP90015: Distributed Systems – Assignment 1

Multi-threaded Subject Enrolment Server (15 marks)

Submission Due: 7th September, 5 PM

1. Overview

In this assignment you will build a client-server system for managing university subject enrolments. A single multi-threaded server holds the shared list of subjects. Any number of client programs can connect to it at the same time to look up a subject's details, enrol a student, withdraw a student, transfer a student between subjects, or update a subject's capacity.

Every connected client acts as a university admin. There is no login and no distinction between clients: any client can perform any of the operations above on any subject.

You will submit two Java programs:

- A server, packaged as SubjectServer.jar
- A client, packaged as SubjectClient.jar

Both are command-line programs, run from a terminal.

This assignment is specifically about two technologies covered in lectures:

- Sockets
- Threads

You must make **explicit** use of both. Sockets and threads must be the lowest level of abstraction you use for network communication and concurrency.

Follow this specification precisely, especially the protocol defined below. Deviating from it, even in ways that seem harmless or that still work when you test your own client against your own server, may cause your submission to be marked incorrectly. If anything in this document is unclear, ask on Ed Discussion rather than guessing.

2. Architecture

- The system follows a client-server architecture in which multiple clients connect to a single multi-threaded server and perform operations concurrently.
- The server may implement thread-per-request, thread-per-connection, or a worker pool. This is a design decision for you to make and justify in your report.
- All communication uses TCP. The reliability guarantees of TCP are sufficient to satisfy the reliability requirement for this assignment.

3. Provided Scaffolding Codes

To let you focus on the core functionalities of this assignment, the following starter codes will be provided to you alongside this specification:

- A **ProtocolMessage** utility class that serializes and deserializes the JSON request/response formats described in the Communication Format and Functional Requirements sections below, so you don't need to hand-write JSON parsing for these exact formats. You are free to modify or extend it, **as long as** your server's protocol output continues to match the specification exactly.
- A **Subject** class to store course-specific data, for you to use as your in-memory representation of a subject.

- **Skeleton files for SubjectClient and SubjectServer.**

This scaffolding covers message formatting, command parsing and data representation only. All socket setup, thread creation and management, locking and concurrency control, and persistence logic must be written by you from scratch. These are the core skills this assignment assesses.

4. Communication Format

All client-server communication is one JSON object per line, terminated by `\n`, sent over the TCP socket. You are responsible for correctly handling partial reads and message boundaries on the receiving end.

Each operation shown in Section 5 specifies its exact request and response format. You must match the operation names, field names, and status strings exactly as shown: renaming or omitting them may cause otherwise-correct functionality to be marked incorrect.

You retain freedom over: internal data structures, persistence file format and layout, error message text (in the *message* field), server architecture, threading model, and locking strategy.

5. Functional Requirements

For each operation below, an admin types a command at the client CLI. The client encodes this as a JSON request message, in the format specified in subsections 5.1 to 5.5, and sends it to the server. The server processes the request and sends back a JSON response.

A response for a given operation may have one of several possible statuses, shown below. The server returns **exactly one** of these for any single request, based on the actual outcome of that operation.

5.1. Query a subject

Looks up a subject and returns its capacity, current enrolment count, and the list of enrolled student IDs. The order of IDs within this list does not matter and is not assessed; it will be compared as a set, not a sequence.

Client CLI: `query <subjectCode>`

Server expects:

```
{"op": "QUERY", "subjectCode": "COMP90015"}
```

Server returns:

```
{"status": "SUCCESS", "data": {  
  "subjectCode": "COMP90015", "capacity": 3,  
  "enrolledCount": 2, "enrolledStudentIds": ["1234567", "7654321"]}}  
  
{"status": "NOT_FOUND", "message": "..."} // subjectCode does not exist
```

5.2. Enrol a student

Adds a student to a subject. If the student is already enrolled in the subject, or the subject is at capacity, no change occurs.

Client CLI: `enrol <subjectCode> <studentId>`

Server expects:

```
{"op": "ENROL", "subjectCode": "COMP90015", "studentId": "1234567"}
```

Server returns:

```
{"status": "SUCCESS"}

{"status": "NOT_FOUND", "message": "..."}    // subjectCode does not exist

{"status": "DUPLICATE_ENROLMENT", "message": "..."}    // already enrolled

{"status": "FULL", "message": "..."}    // course at capacity
```

A successful enrolment must be immediately visible to all other clients querying the course. If the subject does not exist, return NOT_FOUND; this is checked before anything else. If a request would trigger both DUPLICATE_ENROLMENT and FULL at the same time (the student is already enrolled in a subject that is also at capacity), return DUPLICATE_ENROLMENT.

5.3. Withdraw a student

Removes a student from a subject. If the student is not enrolled, no action is taken.

Client CLI: withdraw <subjectCode> <studentId>

Server expects:

```
{"op": "WITHDRAW", "subjectCode": "COMP90015", "studentId": "1234567"}
```

Server returns:

```
{"status": "SUCCESS"}

{"status": "NOT_FOUND", "message": "..."}    // subjectCode does not exist

{"status": "NOT_ENROLLED", "message": "..."}    // student was not enrolled
```

A successful withdrawal must be immediately visible to all other clients querying the subject. If the subject does not exist, return NOT_FOUND; only if the subject exists does NOT_ENROLLED apply.

5.4. Transfer a student between subjects

Atomically removes a student from one subject and adds them to another. No other client's view of either subject may reflect a state where the student is enrolled in both, or in neither, at any point during the operation.

Client CLI: transfer <fromSubjectCode> <toSubjectCode> <studentId>

Server expects:

```
{"op": "TRANSFER", "fromSubjectCode": "COMP90015", "toSubjectCode": "COMP90038",  
"studentId": "1234567"}
```

Server returns:

```
{"status": "NOT_FOUND", "message": "..."}
{"status": "INVALID_REQUEST", "message": "..."}
{"status": "NOT_ENROLLED", "message": "..."}
{"status": "DUPLICATE_ENROLMENT", "message": "..."}
{"status": "FULL", "message": "..."}
{"status": "SUCCESS"}
```

Because several of these conditions can apply to the same request, checks must be applied in this order, stopping at the first that matches:

- NOT_FOUND: if fromSubjectCode does not exist, or if toSubjectCode does not exist (check fromSubjectCode first).
- INVALID_REQUEST: if fromSubjectCode and toSubjectCode are the same course.
- NOT_ENROLLED: if the student is not currently enrolled in fromSubjectCode.
- DUPLICATE_ENROLMENT: if the student is already enrolled in toSubjectCode.
- FULL: if toSubjectCode is already at capacity.
- Otherwise: SUCCESS, and the transfer is applied as described above.

5.5. Update a subject's capacity

Changes a subject's total capacity.

Client CLI: update <subjectCode> <newCapacity>

Server expects:

```
{ "op": "UPDATE_CAPACITY", "subjectCode": "COMP90015", "newCapacity": 120 }
```

Server returns:

```
{ "status": "SUCCESS" }
{ "status": "NOT_FOUND", "message": "..."} // courseCode does not exist
{ "status": "INVALID_REQUEST", "message": "..."} // not a positive integer, or below
current enrolment
```

A successful capacity change must be immediately visible to all other clients querying the subject. newCapacity must be a positive integer; if it is not, or if it is set below the subject's current enrolled count, respond INVALID_REQUEST and leave the capacity unchanged.

For all operations, malformed requests (missing required fields, wrong types, unparseable JSON) must return INVALID_REQUEST or ERROR. The server must not close the connection or terminate on a malformed request: it must send the appropriate response and continue serving further requests on that same connection.

5.6. Other functional requirements

A few additional functional requirements related to the client and server CLI programs are listed in Sections 9.2 and 9.3. Please refer to those sections for more details.

6. Concurrency Requirements

Concurrency control is the central skill this assignment assesses. Your implementation must satisfy all of the following:

- Cross-subject concurrency: operations on different subjects must not block one another.
- Same-subject read concurrency: multiple simultaneous QUERY operations on the same subject must be able to proceed concurrently with each other.
- Write exclusivity per subject: while a write (ENROL, WITHDRAW, TRANSFER, UPDATE_CAPACITY) is in progress on a subject, no other operation, read or write, on that same subject may proceed until it completes.
- Atomicity of the capacity check: ENROL must be atomic with respect to checking and updating the enrolment count. If two clients concurrently attempt to enrol a student into a subject with exactly one

remaining seat, exactly one must succeed and the other must receive FULL. Under no circumstances should both succeed (oversubscription) or both fail when a seat was genuinely available.

- Cross-subject atomicity and deadlock avoidance for TRANSFER: TRANSFER modifies two subjects at once, and must hold write locks on both fromSubjectCode and toSubjectCode for the duration of the operation, excluding all other reads and writes on either subject. Because two clients could concurrently transfer students between the same pair of subjects in opposite directions, your implementation must acquire the two locks in a consistent global order, for example by ascending subjectCode, regardless of which subject is the source and which is the destination for that particular request. Acquiring locks in request order instead (source first, then destination) will deadlock under exactly this scenario, and this will lead to marks deduction.

6.1. The artificial delay argument

The delay argument exists to make lock contention observable. Without an artificial delay, operations on a local machine may complete too quickly for you (or anyone marking your submission) to tell correct concurrent behaviour. The server takes a single delay value in milliseconds as a command-line argument at startup (see the Command-Line Interface section below), applied uniformly to every operation it processes. The placement of this delay in your code is **mandatory**: it must occur immediately after acquiring locks as shown below:

- Acquire the lock(s) appropriate to the operation: a read lock for QUERY; a write lock for ENROL, WITHDRAW, and UPDATE_CAPACITY; both write locks (in the consistent order described above) for TRANSFER.
- Sleep for the configured delay, in milliseconds, while continuing to hold the lock(s).
- Perform the actual read or mutation of shared state.
- Release the lock(s), then send the response.

Lock(s) must be held for the entire duration of the sleep, and must not be acquired only afterwards. This is what makes contention externally observable: if two operations on the same subject genuinely overlap in time, one of them must be measurably blocked waiting for the lock, rather than both completing independently in the delay window. A submission that sleeps before acquiring the lock, or that releases the lock before sleeping, may appear to behave correctly when you test it yourself, but does not actually satisfy the concurrency requirements above, since the blocking behaviour they describe will not actually occur.

7. Failure Model and Persistence

- The data for each subject must be persisted to durable storage after every successful write operation (ENROL, WITHDRAW, TRANSFER, UPDATE_CAPACITY). The persistence file format is your own design choice; it does not need to match the subject data file format described below, but it must be sufficient to fully reconstruct server state on restart.
- If the server process crashes or is restarted, all writes that completed successfully before the crash must still be present when it restarts, restored from the persisted file rather than the original subject data file.
- Errors must be handled gracefully in both client and server: invalid command-line arguments, unreachable server/bad network data, and file I/O errors (missing file, unreadable file, malformed file) must all be caught and reported clearly rather than causing a crash.
- Graceful shutdown of client and server is expected. Re-establishing a dropped TCP connection is not required.

8. Subject Data File Format

The server loads its initial state on startup from a subject data file, passed as a command-line argument (not the persistence file used for crash recovery, which is a separate, server-managed file). The subject data file is a single JSON file containing an array of subject objects, each with exactly the fields shown below:

```
[
  {
    "subjectCode": "COMP90015",
    "capacity": 3,
    "enrolledStudentIds": ["1234567", "7654321"]
  },
  {
    "subjectCode": "COMP90038",
    "capacity": 5,
    "enrolledStudentIds": []
  }
]
```

Your server must enforce the following rules when loading this file and print a clear message and exit with a non-zero status if any of these rules are violated:

- subjectCode values must be unique within the file.
- capacity must be a positive integer.
- enrolledStudentIds may be an empty array, but its length must not exceed capacity.
- The file must be valid JSON matching this structure; a missing file, unreadable file, or malformed JSON must all be reported clearly and must cause the server to exit rather than start with a partially-loaded or empty subject list.

The capacity given for each subject in this file is only its initial value; UPDATE_CAPACITY can change it at runtime, and the persisted (not the original subject data) file is what the server reloads on restart.

9. Command-Line Interface

Both the client and the server are separate command-line programs, launched and operated entirely via a terminal.

9.1. Starting the server and client

```
> java -jar SubjectServer.jar <port> <subject-data-file> <artificial-delay-ms>
> java -jar SubjectClient.jar <server-address> <server-port>
```

Do not hardcode port numbers, addresses, or the delay value. Note that the delay argument belongs to the server only: it governs how the server processes every request it receives, regardless of which client sent it, so there is no separate client-side delay argument.

9.2. Server console

Once running, the server must accept the **following commands** typed into its own console (standard input), in addition to continuously printing an operational log:

- status: prints the current number of active client connections.
- quit: stops accepting new client connections, waits for any in-flight operations to complete, persists final state, and exits cleanly.

The operational log must print one line per completed operation, and must include a timestamp, an identifier for the handling thread, the operation type, the subject code involved, and the resulting status. The exact formatting of this line is your own choice.

9.3. Client interaction

The client connects to the server on startup and then reads commands interactively from standard input, one per line. Each of the five commands shown alongside its operation in Functional Requirements (query, enrol, withdraw, transfer, update) maps to the corresponding request. There is also a sixth command:

- quit: which closes the connection gracefully and exits.

For each command, the client translates it into the corresponding protocol request, sends it, waits for the response, and prints a human-readable summary of the result to the console. The exact wording and formatting of this output is your own design choice. If the user types something that doesn't match one of the known commands (wrong number of arguments, unrecognised command), the client must print a usage message locally and must not send anything to the server.

10. Implementation Language

Java. Use of RMI or JMS is **not permitted**. Use of `java.util.concurrent` thread-safe collections and atomics (e.g. `ConcurrentHashMap`, `AtomicInteger`, `CopyOnWriteArrayList`) as a substitute for writing your own locking logic is **not permitted**, as this assignment is assessing your ability to reason about and implement concurrency control directly, and these collections would bypass that. Standard `java.util.concurrent.locks` primitives such as `ReentrantReadWriteLock` are the intended mechanism and are permitted and encouraged.

11. Assessment

Your submission is assessed in three ways:

- Automated marking of functional correctness and concurrency behaviour, checked directly against this specification.
- A 5-minute oral assessment, where you will navigate your own submitted code and answer questions about your design choices. You should be able to explain and justify every design decision in your submission, particularly your concurrency control mechanism.
- Your written report (see below).

12. Report

Maximum 5 pages (title page not counted). Must include:

- Problem context (brief)
- Component description (max 1 page)
- Class design and interaction diagram (max 1 page)
- Critical analysis (max 2 pages), addressing:
 - Design choices in the parts of the system left open to you (data structures, persistence format, server architecture, thread model)
 - How you implemented “concurrent reads, exclusive per-resource writes,” and specifically how you prevent the last-seat race condition described above
 - How your TRANSFER implementation avoids deadlock when acquiring locks on two subjects at once, and how you chose the lock ordering
 - Pros/cons of your chosen thread model
 - The purpose of the artificial delay argument, exactly where in your code it is applied, and how it affected your design decisions

Do not document anything you have not implemented. This is misconduct and will be penalised severely. Report must be PDF.

13. Submission

Submit individually via LMS (not as one combined archive):

- Report (PDF)
- SubjectClient.jar and SubjectServer.jar
- Source files (.ZIP or .TAR)

14. Academic Integrity

No use of generative AI tools is permitted for this assignment. This includes tools that produce output for inclusion in your submission, modify your work, or guide/direct your work. All work must be your own, per the University's [Academic Integrity policy](#).

The oral component specifically probes your understanding of your own submitted code, and cannot be prepared for without having done the implementation work yourself.

15. Marking

The marking process will be structured by first evaluating whether the assignment (application + report) is compliant with the specification given. This implies the following:

- Correct implementation of the fixed protocol, including the course data file format
- Explicit use of TCP sockets and threads
- Proper concurrency control per the requirements above, including correct placement of the artificial delay and deadlock-free handling of TRANSFER
- Proper error handling
- Persistence and crash recovery
- Report content and format
- Correct behaviour of the server console and client command-line interaction, per the specification above

16. Oral Assessment Schedule and Venue

You are free to develop your system wherever you are most comfortable (at home, on one PC, on your laptop, in the labs, and so on), but keep in mind that this assignment is meant to be a distributed system that works across at least two different machines, in order to properly separate the client from the server.

The oral assessment date, time, and venue will be announced closer to the submission due date. Each tutor will hold 2-3 oral assessment sessions, and you will be required to demonstrate your understanding of your implementation in one of the sessions held by the tutor of the workshop in which you are enrolled.

If you need any clarification on the assignment, ask during tutorials or on Ed Discussion, so that all students can benefit from the answer.

The oral assessment is campus-based; your tutor will announce the schedule closer to the submission date. The oral assessment is mandatory, and non-attendance results in a penalty of -5 marks (see Marking above). You will need to bring your own computer/laptop to the oral assessment.

17. Penalties for Late Submission

Assignments submitted late will be penalised as follows:

- 1 day late: -1 mark
- 2 days late: -2 marks
- 3 days late: -3 marks
- 4 days late: -4 marks
- and so on